

Assignment 2 COSC 2347:

DUE 11:59 pm on 2/24/24 @ 11:59 pm. No late assignments.

Using correct C# style and naming conventions as detailed in the lectures and slides, create a C# program as described below.

I will provide you a test file of “user input” and you will process that file and will produce console output as detailed below.

Part 1: (35 points)

- You will execute a series of ReadLines that correspond to stack operations, ie, requests to push things onto a stack and pop them off. At most there will be 100 requests, but there could be less.
- Your “stack” will be simulated by you storing the items in an array.
- Each request will be a string of the form: push X (where X is a positive int > 0 to be stored on the stack), or pop (where you will need to pop a value from the stack).
- Each time you get a valid push request, output to the console: Pushed (value)
- Each time you get a valid pop request, output to the console: Popped (value)
- Not every request will be in a valid format, so make sure that you handle invalid requests by ignoring them.
- You may be asked to do invalid things like pop more values than are on your stack
- You will process stack operation requests until you receive a request of “(stack end)”.
- It may make your life simpler if you zero out an element in your stack after you pop it off the stack to aid in your debugging.
- I used the string instance method yourString.Split to parse the push commands, but you can also use IndexOf and Substring or other methods.

Part 2: (15 points)

- You will then print the items remaining in your “stack” to the console in the form of:
Stack after processing: 3, 1, 5, 14, 9, 14, 21, 46
- As part of the above process of printing the “stack”, each time you pop an item from your “stack”, you will store it in a different int array.

Part 3: (35 points)

- Now you will execute a series of ReadLines that will correspond to find requests for your new int array.

- Each request will be a string of the form: find X (where X is an int to be located in the array).
- Each time you receive a find request, you will perform a linear search through your array for that item and report which element of the array contained that requested item by printing to the console in the form of:

- Searching array for item X, found it in array position Y.

OR

- Searching array for item X, could not find it.
- In the example above, X is the item you were asked to find, and Y is the array position corresponding to the found array item.
- You will process search request operations until you receive a request of "(find end)".

Part 4: (15 points)

- Print the unsorted array to the console in the form of:

Unsorted Array: 1 5 3 7 11

- Now sort the array (feel free to use the built-in array sort feature which is `Array.Sort(your array)`) and print it again to the console in the form of:

Sorted Array: 1 3 5 7 11

You will:

- 1) Read items from the file using the `Console.ReadLine()` function as if a user were typing the input
- 2) Perform the correct operations as required.
- 3) Console output should be generated through interpolation, not manually constructed strings.
- 4) Output the program's results to the console using the standard `Console.Write` and `Console.WriteLine` commands.
- 5) Your program cannot throw exceptions from conversion results.
- 6) Capture the output screen with the correct results. If your output screen is disappearing before you can capture it, put in a return statement at the end of the program and put a breakpoint on it and then you can capture the output window.
- 7) Turn your complete source code along with the screen capture of your complete program output into Blackboard by the assignment due date. Failure to turn in full source code (in text format) and screen output snapshots showing all of your program output = grade of 0.
- 8) You will NOT use built-in C# language features like lists and stacks to manage or manipulate the stack; you will manage it yourself. If you use built-in language features to manage the stack, you will receive 0 points for the assignment. If you're not sure what's okay, ASK.

- 9) You will NOT use built-in C# language features to search through or manipulate your array, you will manage it yourself. If you use built-in language features on your array, you will receive 0 points for the assignment. If you're not sure what's okay, ASK.
- 10) You will use methods appropriately for your program functions, e.g., you should not have one huge block of code, you should divide your code into methods as appropriate.

Console Redirection:

Reading the file will be easy, we'll use a feature called input redirection and the console input will be read from the file. This is no different than the code that you've been seeing and writing to date.

Console redirection will make your life easy with this program:

Setting up console redirection is easy; in VS go to the debug menu and choose the last option which should be your program's properties. The very first text input box is "command line arguments" and you will copy and paste in all characters after this next colon: `< c:\temp\myinput2.txt`

You will then have to copy myinput2.txt to your computer's c:\temp folder. If you don't have a c:\temp folder, then you will need to create one, and when you execute your code, instead of asking you to input the information from the console, it will read it from this file.

When you no longer wish to input the data from the file, simply remove that line from the command line arguments input box and the program will again accept input from your console.

Macbook users will need to be running the Windows version of Visual Studio in Bootcamp OR running <https://www.jetbrains.com/rider/> OR they can contact me for special code that will simulate the console redirection. VS for Mac does not handle console redirection correctly. If you're running on a Mac with one of the environments above and can't get console redirection to work, contact me.

Notes:

If you are running on a Linux environment or running in another development environment other than VS, you will have to find the comparable options in your development environment.

THERE IS ZERO FILE CODE in this program. If you start down that path, you're heading the wrong direction!

Like all assignments in this class, this is NOT a group project. Writing your code with someone else or sharing code is academic dishonesty and will be dealt with accordingly.

The BIGGEST RISK of this program to your grade is you not carefully following these directions. Check them, double check them, check them again when you're done; your grade will thank you.

As explained in the syllabus, these instructions are subject to verbal change in this class. If you are not paying attention or not here and miss any changes in instructions, that is on you.